Phase 0 – AWS Environment Setup

Objective

The objective of Phase 0 was to prepare the **AWS cloud environment** required for the RetailFlow project. Before processing any data, we needed a secure and organized infrastructure where all project resources could be stored and managed.

What We Did

1. Created an AWS Account

We created an AWS account that would host all the cloud resources required for the project.

Why?

AWS provides scalable cloud services required for building modern data engineering pipelines. Instead of storing data locally, we use cloud services that are secure, reliable, and accessible from anywhere.

---

2. Configured AWS Budget

We created a **monthly AWS Budget** (around **$5**) and enabled budget alerts.

Why?

- To monitor cloud spending.
- To avoid unexpected charges.
- To ensure the project remains within the AWS Free Tier or available credits.

---

3. Selected the AWS Region

We selected the **US East (N. Virginia) – us-east-1** region.

Why?

- Most AWS services are available in this region.

- Lower latency between services.
- Commonly used in tutorials and documentation.
- Cost-effective and well-supported.

---

4. Created the Amazon S3 Bucket

We created the S3 bucket:

retailflow-mahaboob-2026

Why?

Amazon S3 acts as the **Data Lake** for the project. All project files are stored here before being processed.

Instead of storing data on a local machine, S3 provides:

- High durability (99.999999999%)
- Scalability
- Security
- Easy integration with AWS and Databricks

---

5. Created the S3 Folder Structure

Inside the bucket, we created folders such as:

- raw/
- curated/
- quarantine/
- consumption/
- athena-results/
- dq-results/
- clickstream/

Why each folder?

raw/

Stores the original files exactly as received.

Purpose
Acts as the source of truth.

---

curated/

Stores cleaned and transformed datasets.

Purpose
Used for analytics and Delta table creation.

---

quarantine/

Stores invalid or failed records.

Purpose
Separates bad data without affecting good data.

---

consumption/

Stores final datasets used for reporting and dashboards.

Purpose
Business users consume data from here.

---

athena-results/

Stores query results generated by Athena.

Purpose
Athena needs an S3 location to save SQL query outputs.

---

dq-results/

Stores Data Quality reports.

Purpose
Keeps validation results and audit information.

---

clickstream/

Stores customer website/app activity files.

Purpose
Used later for Auto Loader and streaming ingestion in Databricks.

---

6. Installed Required Tools

We prepared the development environment by installing:

- Python
- Boto3

- AWS CLI
- Jupyter/Google Colab
- Required Python libraries

Why?

These tools allow us to:

- Generate data
- Upload files to S3
- Interact with AWS services
- Automate the project

Services Used

Amazon S3

What is it?
AWS object storage service.

Why did we use it?
To build the project's Data Lake and store all datasets securely.

AWS Budgets

What is it?
A service that monitors AWS spending.

Why did we use it?
To prevent unexpected costs during the project.

AWS IAM (used throughout the project)

What is it?
Identity and Access Management service.

Why did we use it?
To securely control permissions for AWS resources. Later phases use IAM roles for Databricks integration.

Deliverables of Phase 0

- AWS Account created
- Budget configured

- AWS Region selected
- S3 bucket (retailflow-mahaboob-2026) created
- Folder structure created
- Development tools installed and configured

Business Value

Phase 0 establishes the cloud foundation for the project. By organizing storage, controlling costs, and preparing the environment, it ensures that all later phases—data ingestion, governance, processing, and analytics—can run reliably and securely.

"In Phase 0, I prepared the AWS environment for the RetailFlow project. I created an AWS account, configured a budget to monitor costs, selected the us-east-1 region, and created an Amazon S3 bucket named retailflow-mahaboob-2026. Inside the bucket, I organized folders such as raw, curated, quarantine, consumption, athena-results, dq-results, and clickstream to support different stages of the data pipeline. I also installed tools like Python, Boto3, and AWS CLI to interact with AWS services. This phase provided a secure, scalable, and well-organized cloud foundation for the entire RetailFlow data engineering project."

Phase 1 – Data Generation & Profiling

Objective

The goal of Phase 1 was to **create realistic retail datasets** and analyze their quality before uploading them to AWS. This ensures the data pipeline starts with reliable and well-understood data.

What We Did

1. Generated Retail Datasets

We created retail data using Python for five business entities:

- **Customers** – Customer information (ID, name, email, address, etc.)
- **Products** – Product details (ID, category, price, stock, etc.)
- **Orders** – Order information (Order ID, Customer ID, Order Date, Status)
- **Order Items** – Products purchased in each order with quantity and price
- **Clickstream** – Customer website/app activity (page views, clicks, timestamps)

Purpose: To simulate real-world e-commerce data.

---

2. Saved the Data

The generated data was stored in different formats:

- **CSV** → Customers and Products (structured master/reference data)
- **JSON** → Orders, Order Items, and Clickstream (transactional and event data)

Why?

- CSV is simple and efficient for structured tables.
- JSON is flexible and better suited for nested or event-based data.

---

3. Performed Data Profiling

We analyzed the generated datasets to understand their quality by checking:

- Number of records
- Data types
- Missing (null) values
- Duplicate records
- Revenue distribution
- Order volume
- Top-selling product categories

Purpose: To identify data quality issues before loading the data into the cloud.

---

4. Created Reports and Visualizations

We generated:

- Data profiling report (data_profile_report.md)
- Revenue distribution chart

- Order volume by day chart
- Top 10 categories by revenue chart
- Null-rate heatmap

Purpose: To visually understand the data and support business analysis.

Why We Performed Phase 1

- To create realistic retail data for the project.
- To understand the structure and quality of the data.
- To identify missing values or duplicates early.
- To ensure only valid data moves into the AWS data lake.
- To establish a strong foundation for the remaining phases.

Technologies Used

- **Python** – Generated synthetic retail datasets.
- **Pandas** – Data manipulation and profiling.
- **NumPy** – Random data generation and numerical operations.
- **Matplotlib** – Created charts and visualizations.
- **JSON & CSV** – Stored the generated datasets.

Deliverables

- customers.csv
- products.csv
- orders_day1.json
- orders_day2.json
- order_items_day1.json
- order_items_day2.json
- clickstream_day1.json
- clickstream_day2.json
- data_profile_report.md
- Revenue distribution chart
- Order volume chart

- Top 10 categories chart
- Null-rate heatmap

Business Value

Phase 1 ensures the project starts with **high-quality, well-understood data**. Detecting issues such as null values or duplicates at this stage reduces errors in later phases and improves the reliability of analytics and reporting.

"In Phase 1, I generated realistic retail datasets for customers, products, orders, order items, and clickstream events using Python. I stored master data in CSV format and transactional/event data in JSON format. After generating the data, I profiled it using Pandas to identify null values, duplicates, and data distributions. I also created reports and visualizations such as revenue distribution and order volume charts. This phase helped ensure that the data was clean and well understood before moving it into the AWS data lake for further processing."

This is the level of detail that is typically expected in interviews: it explains **what you did, why you did it, the tools you used, and the business value**.

Phase 2 – Data Ingestion into Amazon S3

Objective

The objective of Phase 2 was to **upload the generated retail datasets into Amazon S3**, which serves as the project's **Data Lake**. This makes the data centrally available for AWS services and Databricks to process.

What We Did

1. Created a Python S3 Utility

We developed a reusable Python module (s3_ingest) using the **Boto3 SDK** to interact with Amazon S3.

The utility included functions to:

- Upload files
- Download files
- List objects

- Generate pre-signed URLs

Why?

Instead of writing S3 code repeatedly, we created reusable functions. This makes the code cleaner, easier to maintain, and reusable in future projects.

---

2. Configured AWS Credentials

We configured AWS credentials using:

- AWS CLI / Shared Credentials
- Environment variables

We **did not hardcode** the Access Key or Secret Key.

Why?

Hardcoding credentials is a security risk. Using IAM credentials and environment variables follows AWS security best practices.

---

3. Uploaded Retail Datasets to Amazon S3

We uploaded all generated files to the S3 bucket:

Bucket:
retailflow-mahaboob-2026

Files uploaded included:

- customers.csv
- products.csv
- orders_day1.json
- orders_day2.json
- order_items_day1.json
- order_items_day2.json
- clickstream_day1.json
- clickstream_day2.json

Why?

Amazon S3 acts as the centralized storage location where all downstream services (Glue, Athena, Databricks, Lake Formation) can access the data.

---

4. Organized Files in S3

The files were placed into appropriate folders such as:

- raw/
- clickstream/

Why?

A proper folder structure makes the Data Lake organized and simplifies data processing.

For example:

- raw/ contains original source data.
- clickstream/ stores website activity data for Auto Loader.

---

5. Implemented Logging and Error Handling

The upload utility included:

- Structured logging
- Retry mechanism (Exponential Backoff)
- Exception handling

Why?

If network failures or temporary AWS issues occur, the application retries automatically instead of failing immediately. Logging also helps with debugging and monitoring.

---

6. Verified Uploads

After uploading, we listed the objects in the bucket to confirm that all files were successfully stored in Amazon S3.

Why?

Verification ensures that all required files are available before moving to the next phase.

Services Used

Amazon S3

What is it?

Amazon S3 is an object storage service used to build a scalable Data Lake.

Why did we use it?

- Stores raw datasets
- Highly durable and scalable
- Easily integrates with Glue, Athena, Lake Formation, and Databricks

---

Boto3

What is it?

Boto3 is the official AWS SDK for Python.

Why did we use it?

To automate S3 operations such as:

- Uploading files
- Downloading files
- Listing objects
- Managing S3 resources

AWS CLI

What is it?

A command-line tool to interact with AWS services.

Why did we use it?

To configure AWS credentials securely and connect our local environment to AWS.

IAM

What is it?

AWS Identity and Access Management.

Why did we use it?

To securely authorize access to the S3 bucket without exposing credentials.

Deliverables of Phase 2

- Created reusable S3 utility using Boto3
- Configured AWS credentials securely
- Uploaded all retail datasets to Amazon S3
- Organized files into S3 folders
- Added logging and retry logic
- Verified successful uploads

Business Value

Phase 2 establishes a centralized, secure, and scalable Data Lake. By storing all raw data in Amazon S3, the project enables multiple AWS services and Databricks to access the same source of truth for governance, processing, and analytics.

"In Phase 2, I built the data ingestion layer for the RetailFlow project. I developed a reusable Python utility using the Boto3 SDK to upload retail datasets into an Amazon S3 bucket. I configured AWS credentials securely using IAM and environment variables instead of hardcoding keys. The datasets were organized into folders such as raw and clickstream, and I added logging, retry logic, and upload verification to make the ingestion process reliable. This phase created a centralized Data Lake that serves as the source of data for AWS Glue, Athena, Lake Formation, and Databricks."

Phase 3 – Data Lake Organization (Medallion Architecture)

Objective

The objective of Phase 3 was to organize the retail data using the **Medallion Architecture**. Instead of working directly with raw data, we separated the data into different layers (Bronze, Silver, and Gold) so that each stage of processing has a clear purpose. This improves data quality, scalability, and reporting.

What We Did

1. Designed the Medallion Architecture

We divided the data into three logical layers:

- **Bronze Layer**
- **Silver Layer**
- **Gold Layer**

This architecture is widely used in modern Data Lakehouse projects.

Why?

Separating data into layers allows us to:

- Preserve raw data.
- Clean and transform data without affecting the original files.
- Deliver trusted data for business reporting.

---

2. Defined the Bronze Layer

The Bronze layer stores the **raw data** exactly as it is received from the source.

For our project, the Bronze layer included:

- Customers
- Products
- Orders
- Order Items
- Clickstream

Why?

- Keeps the original data unchanged.
- Acts as the backup/source of truth.
- Makes it possible to reprocess data if needed.

---

3. Defined the Silver Layer

The Silver layer contains **cleaned and validated data**.

In this layer we:

- Removed duplicate records.
- Validated data quality.
- Standardized the data.
- Prepared it for analytics.

Why?

Business users should not analyze raw data because it may contain errors or duplicates. Silver provides trusted and consistent data.

---

4. Defined the Gold Layer

The Gold layer stores **business-ready aggregated data**.

Examples:

- Order summary
- Revenue reports
- Sales KPIs
- Dashboard datasets

Why?

Gold tables are optimized for reporting, dashboards, and business intelligence tools. They contain summarized information instead of raw transaction data.

---

5. Planned the Data Flow

The complete data flow was designed as:

Raw Data

|

▼

Bronze Layer

|

▼

Silver Layer

|

▼

Gold Layer

|

▼

Dashboards & Analytics

Why?

Each layer has a specific responsibility:

- **Bronze** → Data Ingestion
- **Silver** → Data Cleansing & Transformation
- **Gold** → Business Analytics

This separation makes the pipeline easier to maintain and troubleshoot.

Services & Technologies Used

Delta Lake

What is it?

Delta Lake is a storage layer built on top of Parquet that provides ACID transactions, versioning, and better reliability.

Why did we use it?

- Ensures data consistency.

- Supports updates and deletes.
- Enables features like Change Data Feed.
- Forms the foundation of the Lakehouse architecture.

Parquet

What is it?

A columnar storage format optimized for analytics.

Why did we use it?

- Faster query performance.
- Better compression.
- Reduced storage cost.
- Efficient for large datasets.

Medallion Architecture

What is it?

A layered data architecture consisting of Bronze, Silver, and Gold.

Why did we use it?

- Organizes data processing into stages.
- Improves data quality.
- Simplifies debugging.
- Supports scalable analytics.

Deliverables of Phase 3

- Designed the Medallion Architecture.
- Defined Bronze, Silver, and Gold layers.
- Planned the complete data transformation flow.
- Identified which datasets belong to each layer.
- Prepared the architecture for Databricks implementation in later phases.

Business Value

The Medallion Architecture ensures that raw data is preserved while clean, validated, and business-ready data is delivered for reporting. It improves reliability, supports future reprocessing, and provides a scalable approach for enterprise data engineering.

"In Phase 3, I designed the Medallion Architecture for the RetailFlow project. I divided the data into Bronze, Silver, and Gold layers. The Bronze layer stores raw data without modification, the Silver layer contains cleaned and validated data after removing duplicates and performing quality checks, and the Gold layer stores aggregated business data used for dashboards and reporting. We used Delta Lake because it provides ACID transactions, versioning, and Change Data Feed support, while Parquet was used for efficient storage and query performance. This layered approach improves data quality, maintainability, and scalability."

Quick Interview Points

Layer	Purpose	Example
Bronze	Store raw data	Orders, Customers, Products
Silver	Clean and validate data	Remove duplicates, standardize records
Gold	Business-ready analytics	Order summaries, KPIs, dashboards

This phase is the **architectural foundation** of the Lakehouse, while the actual implementation of Bronze, Silver, and Gold tables happens later in Databricks (Phase 8).

Phase 4 – Data Quality Validation & Data Curation

Objective

The objective of Phase 4 was to validate the quality of the raw data, identify invalid records, separate them from valid records, and prepare clean datasets for further processing. This ensures that only trusted data moves to the next stages of the pipeline.

What We Did

1. Performed Data Quality Checks

We validated the retail datasets by checking for:

Null (missing) values

Duplicate records

Invalid data

Incorrect formats

Data consistency

Why?

Poor-quality data can produce incorrect reports and business decisions. Data quality checks ensure the data is accurate and reliable.

2. Separated Valid and Invalid Records

After validation:

Valid records were moved to the Curated layer.

Invalid records were moved to the Quarantine layer.

Why?

Instead of deleting bad records, we stored them separately so they could be reviewed and corrected later.

3. Generated Curated Data

The validated datasets were stored in the curated/ folder in S3.

Examples:

Customers

Products

Orders

Order Items

Why?

The curated data is clean and ready for creating Delta tables in Databricks.

4. Generated Data Quality Reports

We created reports that showed:

Number of valid records

Number of invalid records

Duplicate records

Missing values

Validation results

Why?

These reports help monitor the quality of incoming data and provide an audit trail.

5. Stored Data Quality Results

The validation reports were saved in the dq-results/ folder.

Why?

To maintain a history of data quality checks and support future audits or troubleshooting.

Services Used

Amazon S3

Purpose: Stores the validated (curated) data, quarantine data, and data quality reports.

Python

Purpose: Performs validation checks, identifies invalid records, and separates the data into different folders.

Pandas

Purpose: Used to detect duplicates, null values, and perform data transformations.

Folder Structure Used

raw/

Original source data.

curated/

Validated and cleaned data used in the next phases.

quarantine/

Invalid records that failed validation.

dq-results/

Data quality reports and validation summaries.

Deliverables of Phase 4

Performed data quality validation.

Identified null values and duplicate records.

Separated valid and invalid records.

Created curated datasets.

Stored invalid data in quarantine.

Generated and saved data quality reports.

Business Value

Phase 4 ensures that only high-quality, trusted data is used for analytics. By isolating invalid records instead of deleting them, the process maintains data integrity, supports auditing, and improves the accuracy of reports and dashboards.

"In Phase 4, I implemented the data quality layer of the RetailFlow project. I validated the raw datasets by checking for null values, duplicate records, invalid formats, and consistency issues. Valid records were moved to the curated folder, while invalid records were stored in the quarantine folder for future analysis instead of being deleted. I also generated data quality reports and stored them in the dq-results folder. This phase ensured that only clean and reliable data was used in the subsequent Databricks processing stages."

Quick Interview Points

Activity Purpose

Data Validation Ensure data accuracy and completeness

Curated Folder Store clean, validated data

Quarantine Folder Store invalid records for review

DQ Reports Monitor and audit data quality

Business Benefit Improves trust in analytics and decision-making

Outcome: At the end of Phase 4, you had clean, validated datasets in the curated folder, which became the input for the Databricks Bronze layer in the later phases

Absolutely. Based on **your RetailFlow Capstone**, **Phase 5** was about implementing **Data Governance and Security** using **AWS Lake Formation**. This phase ensures that only authorized users can access specific data, especially sensitive information like customer email addresses.

Phase 5 – Data Governance with AWS Lake Formation

Objective

The objective of Phase 5 was to **secure the RetailFlow Data Lake** by implementing data governance using **AWS Lake Formation**. We classified sensitive data, applied access controls, and ensured that different users could only access the data they were authorized to see.

What We Did

1. Registered the S3 Bucket in Lake Formation

We registered our S3 bucket:

retailflow-mahaboob-2026

as a **Data Lake Location** in AWS Lake Formation.

Why?

By default, Lake Formation cannot manage an S3 bucket. Registering it allows Lake Formation to control permissions and governance over the data stored in that bucket.

---

2. Created the RetailFlow Database

We created the database:

retailflow_raw

inside the AWS Glue Data Catalog.

Why?

A database organizes related tables. Instead of accessing files directly from S3, AWS services like Glue and Athena use the Glue Catalog to locate and query datasets.

---

3. Created LF-Tags

We created two LF-Tags:

Sensitivity

Values:

- Public
- Confidential
- PII

Department

Values:

- Analytics
- Engineering

Why?

LF-Tags allow us to classify data and manage permissions based on business rules rather than granting permissions table by table.

---

4. Tagged the Data

We applied tags to datasets and columns.

Example:

Customer email column

Sensitivity = PII

Other business tables

Sensitivity = Public

Why?

Customer email contains personally identifiable information (PII), so it requires stronger access control than public business data.

---

5. Created IAM Personas

We created different user roles such as:

- Data Analyst
- Data Engineer

Why?

Different users require different levels of access.

For example:

Data Analyst

- Can access only Public data.

Data Engineer

- Can access Public, Confidential, and PII data.

This follows the **Principle of Least Privilege**, where users receive only the permissions they need.

---

6. Granted LF-Tag Based Permissions

Instead of granting permissions directly on tables, we granted access based on LF-Tags.

Example:

Data Analyst

Sensitivity = Public

Data Engineer

Sensitivity = PII

Why?

Tag-based permissions are easier to manage and automatically apply to new tables with the same tags.

---

7. Verified Access

We tested access using Athena and confirmed that:

- Data Analysts could not access PII columns.
- Data Engineers could access all required data.

Why?

Testing ensured that governance policies were working correctly before proceeding with analytics.

AWS Services Used

AWS Lake Formation

What is it?

AWS Lake Formation is a service used to build, secure, and govern a Data Lake.

Why did we use it?

- Secure sensitive data
- Control permissions
- Implement data governance
- Manage access using LF-Tags

AWS Glue Data Catalog

What is it?

A centralized metadata repository that stores information about databases and tables.

Why did we use it?

Lake Formation and Athena use the Glue Catalog to discover and manage datasets stored in S3.

IAM

What is it?

AWS Identity and Access Management.

Why did we use it?

To authenticate users and assign appropriate permissions based on their roles.

Amazon S3

What is it?

Cloud object storage service.

Why did we use it?

Stores all project datasets, while Lake Formation governs access to those datasets.

Athena

What is it?

A serverless SQL query service.

Why did we use it?

To verify that governance policies and permissions were applied correctly.

Deliverables of Phase 5

- Registered the S3 bucket as a Lake Formation Data Lake Location.
- Created the retailflow_raw database.
- Defined LF-Tags (Sensitivity and Department).
- Tagged datasets and the customer email column.
- Created IAM personas (Data Analyst and Data Engineer).
- Granted LF-Tag-based permissions.
- Verified access using Athena.

Business Value

Phase 5 ensures that **sensitive customer information is protected** while allowing business users to access only the data relevant to their roles. This improves security, supports compliance requirements (such as GDPR and HIPAA), and simplifies permission management through tag-based access control.

"In Phase 5, I implemented data governance using AWS Lake Formation. I registered the S3 bucket as a Data Lake Location, created the retailflow_raw database in the Glue Data Catalog, and defined LF-Tags such as Sensitivity and Department. I tagged sensitive columns like customer.email as PII and created IAM personas for Data Analysts and Data Engineers. Permissions were granted using LF-Tags instead of individual table permissions, making access management scalable. Finally, I verified the permissions using Athena to ensure users could access only the data authorized for their role."

Quick Interview Points

Task	Why We Did It
Register S3 in Lake Formation	Enable governance over the Data Lake
Create Glue Database	Organize datasets and metadata
Create LF-Tags	Classify data based on sensitivity and department
Tag PII Columns	Protect sensitive customer information
Create IAM Personas	Provide role-based access
Grant LF-Tag Permissions	Simplify and scale access management
Verify with Athena	Confirm that governance policies work correctly

Outcome: At the end of Phase 5, the RetailFlow Data Lake was **secure, governed, and ready for analytics**, with role-based access control ensuring that sensitive data was protected while authorized users could access the information they needed.

Based on **our RetailFlow Capstone**, **Phase 6** focused on **AWS Glue ETL and Amazon Athena**. In this phase, we prepared the curated data for querying by creating metadata in the Glue Data Catalog and then analyzed the data using Athena without moving it into a database.

Phase 6 – AWS Glue & Amazon Athena

Objective

The objective of Phase 6 was to **catalog the curated data stored in Amazon S3 and make it queryable using SQL**. Instead of reading files directly from S3, we created metadata in AWS Glue and used Athena to analyze the data.

What We Did

1. Created AWS Glue Crawlers

We created Glue Crawlers to scan the curated data stored in Amazon S3.

The crawler scanned folders such as:

- customers
- products
- orders
- order_items

Why?

A Glue Crawler automatically:

- Detects files in S3
- Identifies their schema
- Creates metadata in the Glue Data Catalog

Without a crawler, we would have to manually create every table.

---

2. Created Glue Data Catalog Tables

After running the crawler, AWS Glue created tables inside the **retailflow_raw** database.

Example tables:

- customers
- products
- orders
- order_items

Why?

Glue Data Catalog acts like a **metadata repository**. It stores:

- Table names
- Column names
- Data types
- S3 locations

It stores only metadata, **not the actual data**.

---

3. Queried Data Using Athena

We connected Athena to the Glue Catalog and executed SQL queries on the S3 data.

Example queries:

- Total Orders
- Customer Count
- Product Count
- Revenue Analysis

Why?

Athena allows us to query data directly from S3 using standard SQL without loading it into a traditional database.

---

4. Configured Athena Query Results

We configured the S3 folder:

athena-results/

to store Athena query outputs.

Why?

Athena requires an S3 location to save the results of every query execution.

---

5. Validated the Data

We checked that:

- Tables were created successfully.
- Column names matched the source files.
- SQL queries returned correct results.

Why?

Validation ensures that downstream tools like Databricks work with accurate metadata.

AWS Services Used

AWS Glue

What is it?

AWS Glue is a **serverless data integration and metadata management service**.

Why did we use it?

- Discover datasets automatically
- Create metadata

- Build the Data Catalog
- Enable Athena to query data

Glue Crawler

What is it?

A Glue Crawler scans S3 files and automatically creates or updates metadata.

Why did we use it?

Instead of manually defining table schemas, the crawler detects them automatically.

Glue Data Catalog

What is it?

A centralized metadata repository.

It stores:

- Table names
- Column names
- Data types
- File locations

Why did we use it?

Athena and other AWS services use the Glue Catalog to understand the structure of data stored in S3.

Amazon Athena

What is it?

A **serverless interactive SQL query service**.

Why did we use it?

- Query S3 data directly
- No database installation
- Pay only for the data scanned
- Validate our datasets before moving them to Databricks

Amazon S3

Why?

Stores:

- Curated datasets
- Athena query results
- Source files referenced by the Glue Catalog

Deliverables of Phase 6

- Created Glue Crawlers
- Created Glue Data Catalog tables
- Registered metadata for curated datasets
- Configured Athena query result location
- Executed SQL queries in Athena
- Validated data before Databricks processing

Business Value

Phase 6 enables analysts and engineers to query large datasets directly from Amazon S3 using SQL, without maintaining a database. It creates a centralized metadata layer, making data discovery easier and reducing operational costs through serverless querying.

"In Phase 6, I used AWS Glue and Amazon Athena to prepare the curated data for analytics. I created Glue Crawlers to scan the curated datasets stored in Amazon S3. The crawlers automatically detected the schema and created metadata tables in the Glue Data Catalog. I then configured Athena to use the Glue Catalog and executed SQL queries directly on the S3 data. Athena stored the query results in a dedicated S3 folder. This phase allowed us to validate the data and make it easily accessible for analysis without moving it into a traditional database."

Task	**Why We Did It**
Create Glue Crawler	Automatically detect schema from S3
Create Glue Data Catalog	Store metadata for datasets
Query with Athena	Analyze S3 data using SQL
Configure Athena Results	Save query outputs in S3

Task	Why We Did It
Validate Tables	Ensure correct metadata before Databricks

Outcome

At the end of **Phase 6**, the curated data stored in Amazon S3 was **cataloged in AWS Glue, queryable through Athena using SQL, and fully validated**. This prepared the data for the next phase, where **Databricks accessed the same S3 data through Unity Catalog to build the Bronze, Silver, and Gold layers.**

Based on **our RetailFlow project**, **Phase 7** was where we **integrated AWS with Databricks**. The goal was to allow Databricks to securely access the data stored in Amazon S3 without using AWS access keys. This phase established the secure connection between AWS and the Databricks Lakehouse.

Phase 7 – AWS & Databricks Integration6. Amazon Redshift

What we did

- **Created an Amazon Redshift cluster (or Serverless workgroup, depending on your lab).**
- **Connected it to the data pipeline.**
- **Prepared it for analytical queries.**

Role

Amazon Redshift is a cloud data warehouse.

It is used to:

- **Store large amounts of structured, processed data.**
- **Execute very fast SQL queries.**
- **Support business intelligence (BI) tools and dashboards.**

Unlike S3, which stores files, Redshift stores data in database tables optimized for analytics.

Data Flow

S3 Curated Data

|

▼

Databricks (Transform)

|

▼

Gold Tables

|

▼

Amazon Redshift

|

▼

Business Reports / BI

Objective

The objective of Phase 7 was to **securely connect Databricks with the AWS Data Lake**. Instead of using Access Keys and Secret Keys, we configured IAM Roles, Storage Credentials, External Locations, and Unity Catalog to provide secure and governed access to S3.

What We Did

1. Created a Databricks Workspace

We created a Databricks workspace to develop the data pipeline.

Why?

The Databricks Workspace is the environment where we:

- Write notebooks
- Execute Spark code
- Create Delta tables
- Build data pipelines

It acts as the main development platform for data engineering.

---

2. Created a Cross-Account IAM Role

We created an IAM Role named:

dbx-cross-account-role

We configured:

- Trust Policy
- IAM Permissions
- External ID

Why?

Databricks runs in its own AWS account. To access our S3 bucket securely, it must assume an IAM Role in our AWS account. This avoids sharing permanent AWS credentials.

---

3. Configured the Trust Relationship

We updated the IAM Trust Policy using the details provided by Databricks.

The trust policy included:

- Databricks Account ARN
- External ID

Why?

The trust relationship defines **who is allowed to assume the IAM Role**. The External ID protects against unauthorized role assumption.

---

4. Created a Storage Credential

In Unity Catalog, we created a Storage Credential:

dbx-lab-creds

using the IAM Role.

Why?

A Storage Credential securely stores the IAM Role information. Databricks uses it whenever it needs to access data in Amazon S3, eliminating the need to embed credentials in notebooks.

---

5. Created an External Location

We created an External Location pointing to the S3 bucket.

Example:

s3://retailflow-mahaboob-2026/curated/

Why?

An External Location maps an S3 path into Unity Catalog. It allows Databricks to read and write data in S3 while enforcing Unity Catalog permissions.

---

6. Created Unity Catalog Objects

We created:

- Catalog → retailflow
- Schemas:
    - bronze
    - silver
    - gold

Why?

Unity Catalog organizes data using a three-level namespace:

Catalog → Schema → Table

This improves governance and makes data management easier.

---

7. Created a Unity Catalog Volume

We created a Volume:

retailflow.bronze.clickstream_volume

Why?

We used the Volume to store:

- Auto Loader checkpoint files
- Auto Loader schema information

Since DBFS Root is disabled in Databricks Free Edition, Unity Catalog Volumes provide a supported storage location.

---

8. Verified the Connection

We verified that Databricks successfully assumed the IAM Role.

The validation showed:

- Assume Role → Success
- Self Assume Role → Success
- External ID → Success

Why?

This confirmed that Databricks could securely access the S3 bucket.

Services Used

Databricks Workspace

What is it?

The development environment for notebooks, Spark jobs, and data engineering workflows.

Why did we use it?

To build and manage the Lakehouse pipeline.

AWS IAM Role

What is it?

A secure AWS identity with temporary permissions.

Why did we use it?

To allow Databricks to access S3 securely without using Access Keys.

Storage Credential

What is it?

A Unity Catalog object that stores authentication information for cloud storage.

Why did we use it?

To securely connect Databricks to Amazon S3 using the IAM Role.

External Location

What is it?

A Unity Catalog object that links an S3 path to Databricks.

Why did we use it?

To securely read and write data in S3 under Unity Catalog governance.

Unity Catalog

What is it?

Databricks' centralized governance and metadata service.

Why did we use it?

To organize data, manage permissions, apply column masking, and track data lineage.

Unity Catalog Volume

What is it?

Managed storage within Unity Catalog.

Why did we use it?

To store Auto Loader checkpoints and schema files because DBFS was unavailable.

Deliverables of Phase 7

- Created Databricks Workspace.
- Created Cross-Account IAM Role.
- Configured Trust Policy and External ID.
- Created Storage Credential (dbx-lab-creds).
- Created External Location for S3.
- Created retailflow catalog.
- Created bronze, silver, and gold schemas.
- Created Unity Catalog Volume.
- Verified secure connectivity between AWS and Databricks.

Business Value

Phase 7 established a **secure and governed integration** between AWS and Databricks. Instead of exposing AWS credentials, Databricks accessed S3 through an IAM Role, while Unity Catalog centralized metadata, permissions, and governance. This architecture is secure, scalable, and follows enterprise best practices.

"In Phase 7, I integrated AWS with Databricks using Unity Catalog. I created a cross-account IAM Role with a trust policy and external ID, allowing Databricks to securely access the S3 bucket without storing AWS access keys. In Databricks, I created a Storage Credential using the IAM Role, mapped the S3 bucket through an External Location, and organized the Lakehouse using a retailflow catalog with bronze, silver, and gold schemas. Since DBFS was unavailable in the Free Edition, I created a Unity Catalog Volume to store Auto Loader checkpoints and schema information. Finally, I verified that the IAM Role was successfully assumed, confirming secure connectivity between AWS and Databricks."

Task	Why We Did It
Create Databricks Workspace	Development environment for Spark and Delta
Create Cross-Account IAM Role	Secure AWS access without access keys
Configure Trust Policy	Allow Databricks to assume the IAM Role
Create Storage Credential	Secure authentication to S3
Create External Location	Map S3 paths into Unity Catalog
Create Catalog & Schemas	Organize data using a 3-level namespace
Create Unity Catalog Volume	Store Auto Loader checkpoints and schema
Verify Connection	Confirm secure AWS–Databricks integration

Outcome: At the end of **Phase 7**, Databricks was securely connected to your AWS S3 Data Lake and fully prepared to build the **Bronze, Silver, and Gold Delta tables** in the next phase.

Perfect. **Phase 8** is the most important phase because this is where we actually built the **Lakehouse** in Databricks. Below is a concise but interview-ready explanation based on **our actual RetailFlow project**.

Phase 8 – Databricks Lakehouse Implementation

Objective

The objective of Phase 8 was to build the **Databricks Lakehouse** by converting the curated data stored in Amazon S3 into **Delta Tables**, implementing the **Bronze–Silver–Gold architecture**, enabling governance, and preparing the data for analytics and dashboards.

Task 42 – Databricks Environment Setup

What We Did

- Created the Databricks Workspace.
- Connected Databricks with AWS.

- Configured Unity Catalog.
- Verified Storage Credential and External Location.

Why?

Before processing any data, Databricks needed secure access to the S3 bucket.

Outcome: Databricks was ready to read data from S3.

Task 43 – Bronze Layer Creation

What We Did

We read the curated Parquet files from Amazon S3 and created Bronze Delta Tables.

Tables created:

- customers
- products
- orders
- order_items
- orders_dq

Why?

The Bronze layer stores raw business data in Delta format.

Benefits:

- ACID Transactions
- Faster queries
- Versioning
- Time Travel
- Reliable storage

Outcome: Bronze Delta Tables were successfully created.

Task 44 – Auto Loader

What We Did

We configured Auto Loader to read clickstream JSON files.

We created:

- Unity Catalog Volume
- Schema Location

- Checkpoint Location

Auto Loader continuously monitored

s3://retailflow-mahaboob-2026/clickstream/

and loaded new files into

retailflow.bronze.clickstream

Why?

Instead of manually loading files every day, Auto Loader automatically detects and processes new files.

Benefits:

- Incremental ingestion
- Automatic schema handling
- Fault tolerance
- Checkpoint recovery

Outcome: Clickstream data was successfully ingested into the Bronze layer.

Task 45 – Silver & Gold Layer

Silver Layer

What We Did

Created

retailflow.silver.orders

Performed:

- Removed duplicates
- Cleaned records
- Enabled Change Data Feed (CDF)

Why?

Silver contains trusted and validated data.

Gold Layer

What We Did

Created

retailflow.gold.order_summary

Aggregated data based on order status.

Example:

- Delivered
- Pending
- Cancelled
- Shipped

Why?

Gold contains business-ready data used by dashboards and reporting tools.

Outcome: Silver and Gold tables successfully created.

Task 46 – Delta Live Tables (DLT) Pipeline

What We Did

Verified:

- Created a Delta Live Tables (DLT) Pipeline
- Configured the pipeline to process data from the Bronze layer to the Silver layer
- Implemented data quality expectations using:
    - @dp.expect (Warn)
    - @dp.expect_or_drop (Drop invalid records)
    - @dp.expect_or_fail (Fail on critical errors)
- Successfully executed the DLT Pipeline and verified the output table: retailflow.silver.orders_silver_dlt
- Confirmed pipeline execution and data quality metrics in the Databricks Pipeline UI

Why?

Delta Live Tables automates reliable data transformation while enforcing data quality rules. It simplifies pipeline management, improves data reliability, and ensures that only valid, high-quality data is promoted from the Bronze layer to the Silver layer.

Outcome

Task completed successfully. The DLT Pipeline was created, data quality expectations were applied, and the Silver table was successfully generated with validated data.

Task 47 – Unity Catalog Governance

What We Did

Verified:

- Three-level namespace
- Column Masking
- Data Lineage

Applied masking on

customers.email

using

mask_email()

Verified automatic lineage from

Bronze

↓

Silver

↓

Gold

Why?

Column masking protects sensitive customer information, while lineage helps trace how data moves through the pipeline.

Outcome: Task completed successfully.

Task 48 – Delta Sharing

What We Did

Verified:

- Created a Delta Share (retailflow_share)
- Added the Gold table (retailflow.gold.order_summary) to the share
- Verified the shared table was available in the Delta Share
- Attempted to create a Recipient (workspace limitation prevented completion)

Why?

Delta Sharing enables secure, governed data sharing with external users or organizations without copying data. Sharing the Gold layer allows consumers to access curated, analytics-ready data while maintaining centralized governance.

Outcome

Task completed successfully. The Delta Share was created and the Gold table was successfully shared. Recipient creation was not completed due to workspace limitations, but the core Delta Sharing functionality required for the project was implemented and verified**.**

Task 49 – Workflow

What We Did

Prepared the Workflow design.

Workflow sequence:

Auto Loader

↓

Bronze

↓

Silver

↓

Gold

Why?

Workflows automate notebook execution and reduce manual effort.

Status

Workflow design completed.

Execution depends on DLT availability.

Task 50 – Dashboard

What We Did

Prepared the Gold table for reporting.

Used:

retailflow.gold.order_summary

Dashboard shows:

- Order Count
- Order Status Summary
- KPIs
- Business Reports

Why?

Business users need visual insights instead of raw data.

Gold tables provide optimized datasets for dashboards.

Outcome: Gold table is ready for dashboard creation.

Services Used

Service	Purpose
Databricks Workspace	Development environment
Unity Catalog	Metadata & Governance
Storage Credential	Secure S3 authentication
External Location	Maps S3 into Unity Catalog
Volume	Stores Auto Loader schema & checkpoints
Delta Table	Reliable ACID storage format
Auto Loader	Automatic incremental file ingestion
Change Data Feed (CDF)	Tracks row-level changes
Lakeflow / DLT	Managed ETL pipelines
Workflow	Automates notebook execution
SQL Warehouse	Executes SQL queries
Dashboard	Visualizes business KPIs
Delta Sharing	Secure external data sharing

Deliverables of Phase 8

- Databricks environment configured.
- Bronze Delta tables created.
- Auto Loader implemented for clickstream ingestion.
- Silver table created with cleaned data.
- Gold table created with business summaries.
- Change Data Feed enabled.
- Unity Catalog governance configured.
- Column masking applied.

- Data lineage verified.
- DLT notebook prepared.
- Workflow design prepared.
- Gold table ready for dashboard creation.

Phase 9 — Final Integration Report

Task 51 – Final Architecture Diagram

What We Did

Created a comprehensive architecture diagram illustrating the complete RetailFlow data pipeline from data generation to analytics.

Included:

- Google Colab / VS Code for data generation
- Amazon S3 Data Lake (Bronze, Silver, Gold)
- AWS Glue Crawlers and Data Catalog
- Amazon Athena for querying data
- AWS Lake Formation for governance and LF-Tags
- Amazon Redshift Serverless for analytics
- Databricks Workspace with Unity Catalog
- Auto Loader, Delta Live Tables (DLT), and Workflow
- SQL Warehouse and Dashboard for reporting

Why?

The architecture diagram provides a clear visual representation of the end-to-end data engineering solution, showing how data flows through AWS and Databricks while maintaining governance and supporting analytics.

Outcome

Task completed successfully. A complete architecture diagram was created and included in the final report.

Task 52 – Cost Review

What We Did

Prepared a cost review summarizing the cloud resources used during the project.

Included:

- Resources Provisioned
    - Amazon S3
    - IAM Roles and Policies
    - AWS Glue Crawlers
    - AWS Glue ETL Jobs
    - AWS Glue Data Catalog
    - Amazon Athena
    - AWS Lake Formation
    - Amazon Redshift Serverless
    - Databricks Workspace
    - Unity Catalog
    - Storage Credential
    - External Location
    - Delta Live Tables (DLT)
    - Auto Loader
    - Workflow
    - SQL Warehouse
    - Dashboard
- Resources Torn Down
    - Databricks Cluster
    - SQL Warehouse
    - Glue Crawlers
    - Glue ETL Jobs
    - Redshift Serverless
    - Unused S3 resources
- Estimated Monthly Cost at 10× Data Volume

Why?

The cost review helps evaluate cloud resource usage, identify opportunities for cost optimization, and understand how infrastructure costs would scale as data volume increases.

Outcome

Task completed successfully. A cost review was prepared covering provisioned resources, cleanup activities, and expected cost scaling.

Task 53 – Reflection

What We Did

Prepared a reflection on how the solution could be improved for a real production environment.

Suggested improvement:

- Replace scheduled batch processing with an event-driven architecture using Amazon EventBridge or Amazon S3 Event Notifications to automatically trigger the pipeline when new data arrives.

Why?

An event-driven approach reduces processing latency, eliminates unnecessary scheduled executions, improves scalability, and optimizes compute costs compared to fixed schedules.

Outcome

Task completed successfully. A reflection describing a production-ready design improvement was included in the final report.

Business Value

Phase 8 transformed raw retail data into a **governed Lakehouse**. Using Databricks and Delta Lake, the project delivered secure, high-quality, analytics-ready data with automated ingestion, data governance, and a foundation for business dashboards and future workflow automation.

"In Phase 8, I implemented the Databricks Lakehouse architecture. First, I created Bronze Delta tables from the curated Parquet data stored in Amazon S3. Then, I configured Auto Loader to automatically ingest clickstream JSON files into the Bronze layer using Unity Catalog Volumes for schema and checkpoint management. Next, I built the Silver layer by cleaning the Bronze data, removing duplicates, and enabling Change Data Feed. I created the Gold layer by aggregating business metrics into an order_summary table for reporting. To strengthen governance, I applied column masking to the customer email field and verified end-to-end data lineage using Unity Catalog. I also prepared a Lakeflow (DLT) pipeline and workflow for automation. Delta Sharing was attempted but could not be completed because the Databricks Free Edition does not support external OpenSharing. The final outcome was a secure, scalable, and analytics-ready Lakehouse following the Medallion Architecture."

RetailFlow Capstone Project – Overall Explanation

Project Overview

RetailFlow is an **end-to-end Data Engineering project** built on **AWS and Databricks**. The project simulates a real-world retail company where data is generated, stored in a cloud data lake, validated, governed, transformed into analytics-ready datasets, and finally used for reporting and business insights.

The project follows the **Modern Data Lakehouse Architecture** using **Amazon S3, AWS Glue, Athena, Lake Formation, and Databricks with the Medallion Architecture (Bronze, Silver, Gold).**

Business Problem

Retail companies generate large volumes of data from multiple sources, such as:

- Customer registrations
- Product catalogs
- Orders
- Order items
- Website clickstream events

This data is often stored in different formats and may contain duplicate records, missing values, or inconsistent information. Without proper processing, it becomes difficult to generate accurate reports and business insights.

The goal of RetailFlow is to build a secure and scalable pipeline that converts raw retail data into trusted, analytics-ready datasets.

Project Objectives

The project aims to:

- Generate realistic retail datasets.
- Store them in a centralized cloud Data Lake.
- Validate and clean the data.
- Secure sensitive information using governance.
- Build Bronze, Silver, and Gold layers.
- Enable SQL-based analytics.
- Support dashboards and business reporting.

Overall Project Flow

Generate Retail Data

|


Python & Pandas
Amazon S3 (Data Lake)
Data Quality Validation
Lake Formation Governance
AWS Glue Catalog
Amazon Athena
Databricks
Bronze Layer
Silver Layer
Gold Layer

Dashboards & Analytics

Phase-wise Summary

Phase 0 – AWS Setup

- Created AWS account.
- Configured budget alerts.
- Created S3 bucket and folder structure.

Purpose: Prepare the cloud environment.

Phase 1 – Data Generation

- Generated customers, products, orders, order items, and clickstream datasets.
- Performed data profiling.
- Created reports and charts.

Purpose: Create realistic retail data and understand its quality.

Phase 2 – Data Ingestion

- Uploaded datasets into Amazon S3 using Python and Boto3.
- Organized files in appropriate folders.

Purpose: Build the centralized Data Lake.

Phase 3 – Medallion Architecture Design

- Designed Bronze, Silver, and Gold layers.

Purpose: Organize data processing into raw, cleaned, and business-ready layers.

Phase 4 – Data Quality

- Validated records.
- Removed duplicates.
- Separated invalid data into the quarantine layer.
- Generated data quality reports.

Purpose: Ensure only trusted data is processed further.

Phase 5 – Data Governance

- Registered the S3 bucket in Lake Formation.
- Created Glue database.
- Defined LF-Tags.
- Protected PII columns.
- Assigned role-based permissions.

Purpose: Secure the Data Lake and implement governance.

Phase 6 – Glue & Athena

- Created Glue Crawlers.
- Built the Glue Data Catalog.
- Queried data using Athena.

Purpose: Make S3 data queryable using SQL.

Phase 7 – AWS–Databricks Integration

- Created Databricks Workspace.
- Configured Cross-Account IAM Role.
- Created Storage Credential.
- Created External Location.
- Created Unity Catalog and schemas.

Purpose: Securely connect Databricks to Amazon S3.

Phase 8 – Lakehouse Implementation

- Created Bronze Delta tables.
- Configured Auto Loader.
- Built Silver and Gold layers.
- Enabled Change Data Feed.
- Applied column masking.
- Verified lineage.
- Prepared DLT pipeline and Workflow.
- Prepared Gold tables for dashboards.

Purpose: Build a complete Lakehouse for analytics.

Technologies Used

AWS

- Amazon S3
- IAM
- AWS Budgets
- Lake Formation
- AWS Glue
- Athena

Databricks

- Unity Catalog
- Storage Credential
- External Location
- Unity Catalog Volume
- Delta Tables
- Auto Loader
- Change Data Feed (CDF)
- Lakeflow / DLT
- Workflows
- SQL Warehouse
- Dashboard
- Delta Sharing (attempted but limited by Free Edition)

Programming

- Python
- Pandas
- PySpark
- SQL
- Boto3

Final Deliverables

- AWS Data Lake created.
- Retail datasets generated and uploaded.
- Data Quality framework implemented.
- Lake Formation governance configured.
- Glue Catalog and Athena queries created.
- Secure Databricks integration established.
- Bronze, Silver, and Gold Delta tables created.
- Auto Loader configured.
- Change Data Feed enabled.
- Column masking implemented.
- Data lineage verified.
- Gold tables prepared for reporting and dashboards.

Business Value

This project demonstrates how a modern organization can build a secure, scalable, and governed data platform. By combining AWS and Databricks, it enables automated data ingestion, high-quality data processing, strong governance, and business-ready analytics. The Medallion Architecture ensures raw data is preserved, clean data is trusted, and Gold-layer datasets are optimized for reporting and decision-making.

"RetailFlow is an end-to-end Data Engineering project built using AWS and Databricks. We started by generating realistic retail datasets for customers, products, orders, order items, and clickstream events. These datasets were uploaded to an Amazon S3 Data Lake using Python and Boto3. We validated the data quality, separated invalid records into a quarantine layer, and applied governance using AWS Lake Formation with LF-Tags and IAM-based access control. We used AWS Glue Crawlers to create metadata in the Glue Data Catalog and queried the data with Athena. Next, we integrated Databricks with AWS using a cross-account IAM Role, Storage Credential, and External Location. In Databricks, we implemented the Medallion Architecture by creating Bronze, Silver, and Gold Delta tables, configured Auto Loader for incremental clickstream ingestion, enabled Change Data Feed, applied column masking to protect sensitive customer data, and verified data lineage using Unity Catalog. Finally, we prepared the Gold layer for dashboards and reporting, delivering a secure, scalable, and analytics-ready Lakehouse solution."

This explanation clearly covers **what the project does, why each phase exists, the technologies used, and the final business outcome**, which is exactly what interviewers usually look for.